

QUICK REFERENCE CARD

Daily Operations & Troubleshooting

DAILY EXECUTION

Morning (Before Market Open - 9:30 AM)

```
# Navigate to project
cd ~/Downloads/intelligenceos/research-execution

# Activate environment
source venv/bin/activate

# Run research pipeline
./run_daily_research.sh
```

Expected Time: 5-15 minutes **Output:** Signals stored in PostgreSQL

Check Results

```
# View generated signals
psql institutional_intelligence -c "
    SELECT ticker, signal_date, final_score, confidence, p_win, expected_return
    FROM signals
    WHERE signal_date = TODAY
    ORDER BY final_score DESC;
"

# View all agents' activity
tail -100 logs/orchestrator_$(date +%Y%m%d).log
```

COMMAND REFERENCE

Database Queries

```

# Count total signals generated
psql institutional_intelligence -c "SELECT COUNT(*) FROM signals;"

# Top 10 signals by score
psql institutional_intelligence -c "
    SELECT ticker, final_score, confidence, p_win
    FROM signals ORDER BY final_score DESC LIMIT 10;
"

# Signals by sector
psql institutional_intelligence -c "
    SELECT sector, COUNT(*) as signals, AVG(final_score) as avg_score
    FROM signals GROUP BY sector ORDER BY count DESC;
"

# Recent transactions
psql institutional_intelligence -c "
    SELECT c.ticker, ft.transaction_date, ft.shares, ft.price_per_share
    FROM form4_transactions ft
    JOIN companies c ON ft.company_id = c.id
    ORDER BY ft.transaction_date DESC LIMIT 20;
"

```

Agent Status Check

```

# Check Agent 1.1 logs
tail logs/orchestrator_*.log | grep "Agent 1.1"

# Check Agent 2.1 logs
tail logs/orchestrator_*.log | grep "Agent 2.1"

# Count processed companies
psql institutional_intelligence -c "SELECT COUNT(*) FROM companies WHERE last_upd

# Count new signals
psql institutional_intelligence -c "SELECT COUNT(*) FROM signals WHERE created_at

```

TROUBLESHOOTING

Issue: "Database connection failed"

```
# Check PostgreSQL is running
brew services list # macOS
systemctl status postgresql # Linux

# Start PostgreSQL if stopped
brew services start postgresql@15 # macOS
sudo systemctl start postgresql # Linux

# Verify credentials in .env file
cat .env | grep DB_
```

Issue: "ModuleNotFoundError"

```
# Verify virtual environment is active
which python # Should show venv path

# If not, activate:
source venv/bin/activate

# Reinstall requirements
pip install -r requirements.txt
```

Issue: "SEC API rate limit exceeded"

Agent 1.1 automatically handles rate limiting. If you see this:

```
# Check rate limit config
cat configs/parameters/agent_parameters.yaml | grep -A5 "SEC_API"

# Reduce batch size in .env
SEC_API_BATCH_SIZE=25 # was 50
```

Issue: "No signals generated"

```
# Check if data was ingested
psql institutional_intelligence -c "
    SELECT COUNT(*) FROM form4_transactions
    WHERE transaction_date > NOW() - INTERVAL '7 days';
"

# If 0, Agent 1.1 may have failed. Check logs:
cat logs/orchestrator_*.log | grep ERROR
```

Issue: "Low signal accuracy"

Expected for new system. Verify:

```
# Check signal distribution
psql institutional_intelligence -c "
    SELECT
        confidence,
        COUNT(*) as count,
        AVG(expected_return) as avg_return
    FROM signals
    GROUP BY ROUND(confidence::numeric, 2)
    ORDER BY confidence;
"

# Verify scoring parameters are correct
cat configs/parameters/agent_parameters.yaml
```

MONITORING DASHBOARD (Manual)

Create `monitor.sh`:

```
#!/bin/bash

while true; do
    clear
    echo "=====
    echo "INSTITUTIONAL INTELLIGENCE OS"
    echo "Live Monitor - $(date)"
    echo "=====
    echo ""

    # Recent signals
    echo "[Recent Signals]"
    psql institutional_intelligence -c "
        SELECT ticker, final_score, confidence FROM signals
        ORDER BY created_at DESC LIMIT 5;
    "

    echo ""
    echo "[Stats]"
    psql institutional_intelligence -c "
        SELECT
            COUNT(*) as total_signals,
```

```

        AVG(final_score) as avg_score,
        AVG(confidence) as avg_confidence,
        AVG(p_win) as avg_win_prob
    FROM signals;
"

echo ""
echo "[Latest Execution]"
tail -1 logs/orchestrator_*.log

echo ""
echo "Refresh in 30 seconds (press Ctrl+C to exit)..."
sleep 30
done

```

Run it:

```

chmod +x monitor.sh
./monitor.sh

```

AGENT-SPECIFIC COMMANDS

Agent 1.1: SEC Form 4 Scraper

```

# Run just Agent 1.1 (data collection only)
python agents/data_ingestion/agent_1_1_sec_scraper.py

# Run with specific tickers
python -c "
from agents.data_ingestion.agent_1_1_sec_scraper import SecForm4ScraperAgent
import asyncio
import os

db_conn = f"postgresql://{os.getenv('DB_USER')}:{os.getenv('DB_PASSWORD')}@{os.g
agent = SecForm4ScraperAgent(db_conn)
asyncio.run(agent.run(sample_tickers=['NVDA', 'META', 'NFLX']))
"

```

Agent 2.1: Insider Cluster Detector

```

# Run just Agent 2.1 (signal generation only)

```

```
python agents/signal_generation/agent_2_1_cluster_detector.py

# Test cluster scoring
python -c "
from agents.signal_generation.agent_2_1_cluster_detector import InsiderClusterDet
import os

db_conn = f"postgresql://{os.getenv('DB_USER')}:{os.getenv('DB_PASSWORD')}@{os.g
agent = InsiderClusterDetectorAgent(db_conn)

# Detect clusters for specific ticker
clusters = agent.detect_clusters('AAPL')
for c in clusters:
    print(f"{c['ticker']}: Score={c['final_score']:.1f}\n")
"
```

WEEKLY MAINTENANCE

Every Monday

```
# Backup database
pg_dump institutional_intelligence > backups/intelligence_os_$(date +%Y%m%d).sql

# Archive old logs
mkdir -p logs/archive/$(date +%Y/%m)
mv logs/orchestrator_*.log logs/archive/$(date +%Y/%m)/

# Check agent performance
psql institutional_intelligence -c "
SELECT
    agent_name,
    COUNT(*) as executions,
    AVG(duration_ms) as avg_duration,
    SUM(CASE WHEN status = 'error' THEN 1 ELSE 0 END) as errors
FROM agent_logs
WHERE timestamp > NOW() - INTERVAL '7 days'
GROUP BY agent_name;
"
```

Every Month

```
# Full database maintenance
```

```
psql institutional_intelligence -c "VACUUM ANALYZE;"

# Archive signals older than 30 days (optional)
psql institutional_intelligence -c "
    DELETE FROM signals
    WHERE created_at < NOW() - INTERVAL '30 days'
    AND validation_status = 'rejected';
"

# Generate monthly report
python analyze/monthly_report.py
```

PERFORMANCE EXPECTATIONS

Agent 1.1 (SEC Scraper)

- **Companies/run:** 5-20
- **Form 4 filings/run:** 20-50
- **Transactions/run:** 100-500
- **Duration:** 5-30 seconds
- **Success rate:** >99%

Agent 2.1 (Cluster Detector)

- **Companies analyzed:** 5-20
- **Clusters detected:** 5-20
- **Signals generated:** 2-8
- **Duration:** 2-10 seconds
- **Typical scores:** 10-13 (threshold is 10)

Full Pipeline

- **Total duration:** 10-60 seconds (research mode, no trading)
 - **Database writes:** 500-2000 records
 - **Signals per run:** 2-8
-

CONFIGURATION CHANGES

Change minimum signal score threshold

```
# In configs/parameters/agent_parameters.yaml
SIGNAL_THRESHOLDS:
  minimum_score: 10 # Change this (was 10)
  minimum_confidence: 0.70
```

Then restart:

```
./run_daily_research.sh
```

Change sector multipliers

```
# In configs/parameters/agent_parameters.yaml
SECTOR_MULTIPLIERS:
  Technology: 1.15 # Modify any sector weight
  Healthcare: 1.12
```

Add new companies to monitor

```
# Add companies to database
psql institutional_intelligence -c "
  INSERT INTO companies (ticker, cik, name, sector)
  VALUES ('NVDA', '0001318605', 'NVIDIA', 'Technology')
  ON CONFLICT DO NOTHING;
"

# Then Agent 1.1 will automatically include them
```

NEXT AGENT IMPLEMENTATION

When ready to add more agents:

Agent 2.2: Flow Analyzer

```
# Copy template
cp agents/signal_generation/agent_2_2_flow_analyzer.py.template \
```

```
agents/signal_generation/agent_2_2_flow_analyzer.py

# Implement the class and test
python agents/signal_generation/agent_2_2_flow_analyzer.py

# Add to orchestrator (orchestration/orchestrator.py line ~42)
```

Agent 2.3: Chop Filter

```
# Copy template
cp agents/signal_generation/agent_2_3_chop_filter.py.template \
  agents/signal_generation/agent_2_3_chop_filter.py

# Implement and test
python agents/signal_generation/agent_2_3_chop_filter.py
```

Agent 3: Probability Scorer

```
# Copy template
cp agents/prediction/agent_3_probability_scorer.py.template \
  agents/prediction/agent_3_probability_scorer.py

# Implement and add to orchestrator
```

SUCCESS CHECKLIST

- ✓ PostgreSQL running
- ✓ Database created (institutional_intelligence)
- ✓ Virtual environment active
- ✓ .env file configured with API keys
- ✓ Agent 1.1 runs without errors
- ✓ Agent 2.1 runs without errors
- ✓ Signals stored in database
- ✓ Daily pipeline runs successfully
- ✓ Logs created and updated

Once all checked:

- ✓ Begin collecting signal outcomes (4+ weeks)
 - ✓ Validate accuracy manually
 - ✓ Implement remaining agents
 - ✓ Test forward predictions
 - ✓ Prepare for execution phase
-

QUICK LINKS

- **Main docs:** `SETUP_GUIDE_COMPLETE.md`
 - **Architecture:** `MULTI_AGENT_AUTOMATION_SYSTEM.md`
 - **Deployment plan:** `DEPLOYMENT_ROADMAP_4_WEEKS.md`
 - **Database:** `psql institutional_intelligence`
 - **Logs:** `logs/orchestrator_YYYYMMDD.log`
 - **Signals:** `SELECT * FROM signals;`
-

Status: Your system is research-ready. Run daily and collect outcomes.